

Ben-Gurion University of the Negev
Faculty of Computer and Information Science
Deep Learning – Assignment 2

Purpose of the assignment

Enabling students to experiment with building a convolutional neural net and using it on a real-world dataset and problem. You will design and train face verification systems based on Siamese / metric-learning principles. The goal is to not only to produce a working network, but also to run a fair, well-controlled empirical comparisons and carefully analyze what your results actually show.

Submission instructions

- **Due date:** 15.5.2026
- The assignment is to be carried out using PyTorch.
- **Submission in pairs only.** Only one copy needs to be uploaded to Moodle.
- Plagiarism of any kind (including copying from public Siamese / FaceNet / metric-learning repositories) is forbidden. Use of small standard utilities – data loaders, plotting helpers, FLOP counters – is permitted if explicitly cited.
- The entire project is submitted as a single zip file containing the report (PDF) and the code. It is the students' responsibility to verify that the file is valid.
- **The report** must be authored as a .docx file (Calibri, 12pt, 2.5 cm margins) and submitted as PDF. The body of the report is limited to **8 pages**. Figures, full result tables, embedding visualizations, additional ablations, and per-subgroup breakdowns belong in an appendix and must be referenced from the body.

Background

Your starting point is the paper *Siamese Neural Networks for One-shot Image Recognition* (Koch, Zemel & Salakhutdinov, 2015):

<https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf>. Review it before starting.

Note the limitations of the original setup: a single L1-distance head trained with binary cross-entropy and a hand-crafted CNN backbone. Modern face verification differs in loss/sampling, backbone, and evaluation protocol, and you will explore each of those differences directly in this assignment.

Dataset

Use the aligned LFW variant (LFW-a) from

<https://talhassner.github.io/home/projects/lfwa/index.html>. Use the train/test splits provided in pairsDevTrain and pairsDevTest. The test split contains identities not seen during training; do not violate this. You may carve a validation split from the training data, but it is important that you document and justify your choices.

In your report, include the standard dataset analysis: number of identities, images per identity (distribution, not just mean), and total pair counts per split.

Required experiments

You will run two coordinated comparisons plus a baseline. The same training pairs, the same test pairs, and the same one-shot evaluation episodes must be used across all configurations within each experiment. The point is not to find a single winning system but to learn what each axis contributes.

Experiment 1 – Loss function

Hold the backbone fixed (use a Koch-style CNN as the baseline) and compare three losses:

1. **Koch-style:** shared CNN → element-wise L1 distance → sigmoid → binary cross-entropy on the pair label. This reproduces the paper's setup.
2. **Contrastive loss** (Hadsell, Chopra & LeCun, *Dimensionality Reduction by Learning an Invariant Mapping*, CVPR 2006: <https://yann.lecun.com/exdb/publis/pdf/hadsell-chopra-lecun-06.pdf>) on L2 distance in embedding space, with a margin you tune.
3. **Triplet loss with semi-hard negative mining** (introduced in the FaceNet paper, Schroff et al., 2015). You must implement the mining yourself; random triplet sampling is not an acceptable triplet baseline, though you should also report it as an ablation to show what mining buys you.

Discuss in the report how each loss shapes the embedding, the role of the margin, why mining matters for the triplet loss, and how you chose your margins.

Short tutorials on both approaches (including the needed equations):

- **PyTorch tutorial on Siamese networks with contrastive loss:** <https://perso.esiee.fr/~chierchg/deep-learning/tutorials/metric/metric-1.html>. Implements the Hadsell et al. contrastive loss in PyTorch on MNIST and evaluates the learned embeddings with k-NN and t-SNE – the same setup requested in this assignment .

- **Triplet loss and online triplet mining** (Moindrot): <https://omoindrot.github.io/triplet-loss>. The code is in TensorFlow but fully applicable to PyTorch.

Experiment 2 – Backbone

Fix the loss to whichever performed best in Experiment 1, and compare two backbones:

1. The Koch-style CNN you trained earlier.
2. A ResNet-18 trained from scratch on LFW-a (no ImageNet weights – see note below).

Both networks need to be trained “from scratch”: The point of this experiment is to compare what each architecture “buys” you on this task – residual connections vs. plain stacking. If you start from pre-trained weights (e.g., ImageNet) for the ResNet but train Koch from random init, you are no longer comparing architectures, you are comparing pretraining. ResNet would then win for the wrong reason. The pretrained ResNet has its own dedicated slot in Experiment 3.

Important: adjusting the networks’ parameter count. The original Koch architecture has roughly 38M parameters, almost entirely due to its first fully-connected layer ($\approx 9216 \rightarrow 4096$), which alone accounts for ~ 37.7 M parameters. ResNet-18 has ~ 11.7 M parameters. If you compare the two networks as published, you are not comparing residual connectivity

against plain stacking – you are mostly comparing capacity, and Koch will look strong simply because it is much larger.

To isolate the architectural variable, you must bring the parameter counts within **roughly $\pm 20\%$ of each other before training**. The recommended way to do this is:

- **Use ResNet-18** (~11.7M parameters) as your modern backbone – not a smaller ResNet-10 / ResNet-12 variant. The slightly larger ResNet is chosen specifically so that less surgery is needed on Koch.
- **Reduce Koch's first fully-connected embedding layer from 4096 to 1024 units**. This brings Koch to roughly 10–11M parameters, which is within ~10% of ResNet-18.
- **Standardize the embedding dimension across both networks**. Use the same final embedding dimension (e.g., 128 or 256) at the output of both backbones, attached as a small linear head. This keeps the metric-learning head fixed and isolates the backbone.

You are free to choose slightly different numbers (e.g., a different reduction factor for the FC layer) as long as you (i) report the actual parameter counts of both networks and (ii) verify and state that the $\pm 20\%$ matching constraint is met. Document this clearly in the report.

Reporting for Experiment 2. Report parameter counts, FLOPs (see Tools and instrumentation below), and training wall-clock time for both backbones. Discuss what residual connectivity buys you on this dataset at this scale, and whether the ranking matches what you would have predicted from the lectures.

Experiment 3 – Frozen pretrained-features baseline

As a sanity floor, take a frozen ImageNet-pretrained ResNet-18 (no training, no fine-tuning), extract embeddings on aligned LFW-a crops, and evaluate verification with cosine similarity. Report the same metrics on exactly the same test pairs and one-shot episodes you used for your trained models.

If a model from Experiment 1 or Experiment 2 fails to beat this baseline, treat that as a finding and discuss it honestly. Do not hide it.

Required evaluation protocol

For every model in Experiments 1–3, report all of the following on the same test pairs and the same set of one-shot episodes:

- **Verification accuracy** at the threshold you chose, with a written justification of how you chose it.
- **ROC curve and AUC**. Plot ROC curves for all models on a single figure to make the comparison legible.
- **N-way one-shot accuracy** in the style of Koch et al., for $N \in \{2, 5, 20\}$. The same set of episodes must be used across all models.
- **Training and validation loss curves** and convergence wall-clock time.

A single summary table comparing all models on all metrics is required. For the best model in each experiment, report mean $\pm$ std across at least 3 random seeds.

Required embedding analysis

For your best model overall:

- A 2D projection (t-SNE or UMAP) of test-set embeddings, colored by identity. Sample 20–30 identities for legibility.
- A short analysis of intra-class and inter-class embedding distances on the test set.
- A short discussion of what these tell you about the geometry the loss has induced – separation, compactness, and any structure you did not expect.

Tools and instrumentation

Counting parameters and FLOPs

For the backbone comparison in Experiment 2 you must report parameter counts and FLOPs.

The recommended library is **thop** (a.k.a. pytorch-OpCounter):

<https://github.com/Lyken17/pytorch-OpCounter>. It installs in one step (`pip install thop`) and gives you the numbers in two lines:

```
from thop import profile
macs, params = profile(model, inputs=(dummy_input,))
```

You may use a different tool (e.g., `fvcore`, `ptflops`, `torch_flops`) if you prefer, but you must state explicitly which library you used and on what input size. Different libraries can give slightly different numbers because they account for different sets of operations.

MACs vs. FLOPs – be careful

`thop` reports MACs (multiply-accumulate operations), not FLOPs. One MAC is conventionally counted as two FLOPs (one multiplication and one addition). Many papers and many of these libraries are sloppy about the distinction. In your report, state explicitly whether you are reporting MACs or FLOPs, and if you are reporting FLOPs, state the conversion factor you used. Comparisons across your two backbones must use the same convention.

Compute budget and fairness constraints

To keep the comparisons honest, every configuration in Experiments 1 and 2 must use the same compute budget. Specifically:

- The same number of training epochs, or the same number of FLOPs (state which you adopted).
- The same hyperparameter search budget per configuration – declare K , the maximum number of trials you allow yourselves per configuration, and stick to it.
- The same input pipeline and augmentation policy, unless an augmentation difference is itself the variable being studied.
- The same evaluation episodes.

Cherry-picking a longer schedule for one configuration to "make it work" is not the goal of the assignment and will be penalized. Documenting that one configuration failed to converge under the shared budget is itself a legitimate finding – write it up.

Reporting requirements

In addition to the experimental content, the report must include:

- **A clear statement of your compute budget and fairness constraints**, placed before any results.
- **A failure-case analysis** for your best model – at least 6 misclassified pairs (3 false accepts, 3 false rejects), each with a hypothesis about why the model failed (pose, illumination, occlusion, age, image quality).
- **A "threats to validity" paragraph**: what would change your conclusions, what you did not have time to control for, and what your seed and budget choices may have hidden.

As before: if asked to recreate your work, a peer should be able to do so from your report alone.

Grading

Indicative weights:

- Correctness of implementation and code quality – 20%
- Experimental setup, fairness, and methodology – 25%
- Quality of analysis (embedding, evaluation, failure cases, threats to validity) – 25%
- Clarity, completeness, and reproducibility of the report – 20%
- Originality and depth in the discussion – 10%

Performance numbers themselves are not the key for grading. A pair that runs honest, well-controlled experiments with mediocre numbers will score higher than a pair with strong numbers and weak methodology.